

Week 1

Number Representation & Error Analysis

IEEE 754 · Machine Epsilon · Forward/Backward Error · Catastrophic Cancellation · Conditioning vs Stability

Name: Dr Zita Lifelo
Mrs Daka



Week 1 — Lecture Roadmap

1

IEEE 754 Floating-Point Standard

Bit layout · Single, double, extended precision · Special values · Rounding modes

2

Machine Epsilon, ULP & Rounding

ϵ_{mach} · Units in the last place · What 'correct rounding' means in hardware

3

Error Taxonomy

Absolute · Relative · Forward · Backward — definitions and examples

4

Catastrophic Cancellation

Loss of significance · The quadratic formula failure · Stable reformulations

5

Conditioning vs. Stability

A problem property vs. an algorithm property — the distinction students most often miss



Why Floating-Point Arithmetic Matters

Real failures. Real consequences.

1991 Patriot Missile Failure — Dhahran, Saudi Arabia

A time counter was stored as a 24-bit integer, then multiplied by $1/10$ (0.1 in binary is non-terminating). After 100 hours of operation, the accumulated rounding error was 0.34 seconds — enough for the missile to miss its target by 600 metres. 28 soldiers killed.

1996 Ariane 5 Rocket Explosion — Kourou, French Guiana

A 64-bit floating-point number (horizontal velocity) was converted to a 16-bit signed integer. The value exceeded the 16-bit maximum: overflow exception, unhandled error, self-destruct. \$370 million destroyed 37 seconds after launch.

1982 Vancouver Stock Exchange Index Error

The index was repeatedly truncated (not rounded) after each transaction. Over 22 months, the cumulative truncation error dragged the reported value from 1098.892 to 524.811 — nearly a 50% discrepancy from the true value.

What is floating-point?

A system for representing a large range of real numbers using a fixed number of binary digits. The 'point' floats — the exponent slides the significand to any magnitude.

Adopted in 1985

IEEE Std 754-1985 standardised floating-point across all processors. Revised 2008, 2019.

Not the real numbers

Only $\sim 2^{64}$ distinct values exist in double precision — a tiny fraction of the reals.

Base-2 scientific notation

Every number is: $(-1)^s \times 1.\text{mantissa} \times 2^{(\text{exponent}-\text{bias})}$

Formula

General form:

$$\text{value} = (-1)^s \times M \times 2^E$$

where:

s = sign bit (0 = positive)
 M = significand (1.mantissa)
 E = biased exponent

Single precision example:

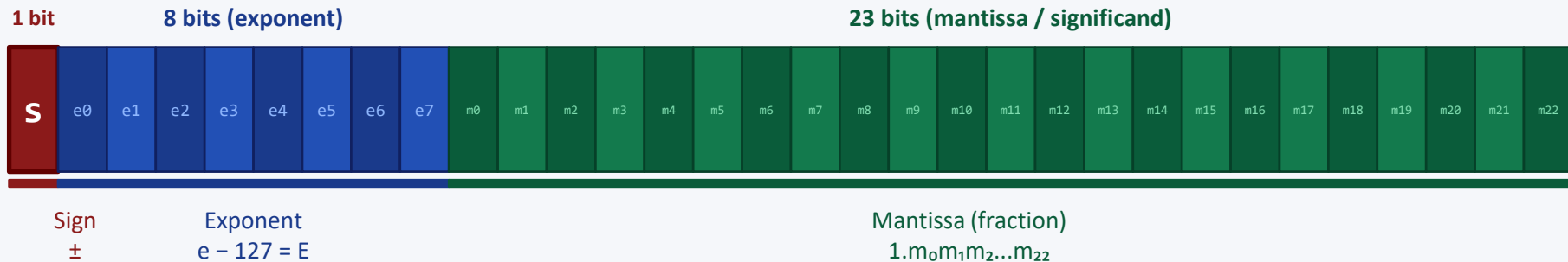
$$E = \text{stored_exp} - 127 \quad (\text{bias}=127)$$

In Python:

```
import struct
bits = struct.pack('f', 1.0)
hex(int.from_bytes(bits, 'little'))
```

IEEE 754 — Bit Layout: Single Precision (float32)

SECTION 1 · IEEE 754



Total bits

32

= 4 bytes

Decimal digits

~7.2

significant

Max value

$\sim 3.4 \times 10^{38}$

FLT_MAX

Machine ϵ

$\approx 1.19 \times 10^{-7}$

2^{-23}

Property	float32 (single)	float64 (double)	float80 (extended)
Total bits	32	64	80 (Intel x87)
Sign bit	1	1	1
Exponent bits	8	11	15
Mantissa bits	23	52	63 + explicit 1
Exponent bias	127	1023	16383
Decimal digits	~7.2	~15.9	~18.5
Machine ϵ	$\approx 1.19 \times 10^{-7}$	$\approx 2.22 \times 10^{-16}$	$\approx 1.08 \times 10^{-19}$
Max value	$\sim 3.4 \times 10^{38}$	$\sim 1.8 \times 10^{308}$	$\sim 1.2 \times 10^{4932}$
numpy dtype	np.float32	np.float64	np.longdouble*
C type	float	double	long double



* np.longdouble behaviour is platform-dependent. On most x86-64 Linux systems it maps to 80-bit extended. On Windows/macOS it is often identical to float64.

+Inf / -Inf

Exp=11111111, Mant=0

Result of overflow or division by zero. Propagates through arithmetic: $\text{Inf} + 1 = \text{Inf}$. Not an error by itself!

NaN

Exp=11111111, Mant $\neq$ 0

Not a Number. Result of $0/0$, $\sqrt{-1}$, $\text{Inf}-\text{Inf}$. NaN is contagious: any arithmetic with NaN returns NaN. $\text{NaN} \neq \text{NaN}$ is TRUE.

-0

Sign=1, Exp=0, Mant=0

Negative zero. $0.0 == -0.0$ evaluates to True, but $1/0.0 = +\text{Inf}$ and $1/(-0.0) = -\text{Inf}$. Mostly irrelevant — but can surprise.

Denormals

Exp=00000000, Mant $\neq$ 0

Sub-normalised numbers filling the gap near zero. No implicit leading 1. Slower on hardware ('gradual underflow'). May trigger FP exceptions.

```
>>> float('inf') + 1 → inf      | >>> float('nan') == float('nan') → False      | >>> 1.0/0.0 → ZeroDivisionError (Python  
raises; C returns Inf)
```

Machine Epsilon (ϵ_{mach})

ϵ_{mach} is the smallest positive number ϵ such that:
 $1.0 + \epsilon \neq 1.0$
in the floating-point system.

For float64: $\epsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$

```
import numpy as np

eps = np.float64(1.0)
while np.float64(1.0) + eps != np.float64(1.0):
    eps /= 2.0
eps *= 2.0 # last value that still differed

print(f'Manual: {eps:.6e}') # 2.22e-16
print(f'numpy: {np.finfo(np.float64).eps:.6e}')
```

Python

ULP — Unit in the Last Place

1 ULP is the gap between a floating-point number and the next representable number. The gap varies with magnitude — it is small near zero, large near Inf.

```
np.spacing(1.0) → 2.22e-16 # 1 ULP at 1.0
```

IEEE 754 Rounding Modes

1 Round to nearest (ties to even)

2 Round toward $+\infty$ (ceiling)

3 Round toward $-\infty$ (floor)

4 Round toward zero (truncate)

Default is mode 1 (round to nearest, ties to even).



SECTION 2

Error Analysis

Absolute · Relative · Forward · Backward

"The purpose of computing is insight, not numbers." — Richard Hamming

Absolute Error

$$|x^{\wedge} - x|$$

The raw magnitude of the error. Units match the quantity.

True: 3.14159265 | Computed: 3.14159 | Absolute: 2.65×10^{-6}

Relative Error

$$|x^{\wedge} - x| / |x|$$

Error as a fraction of the true value. Dimensionless. Preferred for comparing across scales.

True: 3.14159265 | Relative: $8.43 \times 10^{-7} \approx 0.000084\%$

Forward Error

$$|f(x^{\wedge}) - f(x)|$$

How far is the computed output from the true output?
Measures the damage done.

We asked for $f(x)$. We got $f(x^{\wedge})$. Forward error = how wrong the answer is.

Backward Error

$$|x^{\wedge} - x^*|$$

where $f(x^*) =$
true output

What is the smallest input perturbation that would make our answer exact? Measures how well we solved a nearby problem.

We computed $\hat{y}$. Backward error = how small a Δx makes $f(x + \Delta x) = \hat{y}$ exactly.

Problem: evaluate $f(x) = \sqrt{x}$ at $x = 2$ using finite-precision arithmetic.

FORWARD ERROR → how wrong is the answer?

True answer:

$$f(2) = 1.41421356237...$$

float64 computed:

$$\mathbf{f(2) = 1.4142135623730951}$$

Absolute forward error:

$$|f(2) - \mathbf{f(2)}| \approx 2.4 \times 10^{-17}$$

Relative forward error:

$$\approx 1.7 \times 10^{-17} \text{ (excellent!)}$$

BACKWARD ERROR → what problem did we solve?

We computed $\mathbf{f(2) = 1.4142135623730951}$

What input x^* would give this exactly?

$$\sqrt{x^*} = \mathbf{f(2)} \rightarrow x^* = (\mathbf{f(2)})^2$$

$$x^* \approx 2.0000000000000004$$

Backward error:

$$|x^* - 2| \approx 4.4 \times 10^{-16}$$

Insight: $\sqrt{}$ in IEEE 754 is backward stable — the result is the exact $\sqrt{}$ of a number within 1 ULP of the true input.



A backward-stable algorithm produces a result that is the exact answer to a slightly perturbed problem. This is the gold standard in numerical computing.

What is cancellation?

When two nearly equal floating-point numbers are subtracted, the leading significant digits cancel and the result is dominated by round-off in the trailing digits. The relative error can become catastrophically large.

a	=	1.2345678
b	=	1.2345671

a - b = 0.0000007



Only 1 significant digit remains!

The mechanism:

- 1 Both inputs are rounded to float
- 2 Leading digits are equal — they cancel exactly
- 3 Trailing digits contain rounding noise
- 4 Result is mostly noise — huge relative error

```
Python
# Demonstration
a = np.float32(1.2345678)
b = np.float32(1.2345671)

naive = a - b
# Expected: 7e-7
# Got:      ~8.34e-7 (wrong!)
```

$ax^2 + bx + c = 0$ with $a=1$, $b=-10000$, $c=1$

True roots: $x_1 \approx 10000.0001000$ and $x_2 \approx 0.0001000000010\dots$

NAIVE — standard formula

```
• • • Python
import numpy as np
a, b, c = 1.0, -10000.0, 1.0
disc = np.sqrt(b**2 - 4*a*c)

x1 = (-b + disc) / (2*a)
x2 = (-b - disc) / (2*a)

# x1 = 10000.0001      ✓ good
# x2 = 0.0             ← WRONG!
# (b+disc ≈ 0 cancellation)
```

✗ x_2 loses all precision — cancellation in $(-b - \text{disc})$ when b is large.

STABLE — reformulated

```
• • • Python
def stable_roots(a, b, c):
    disc = np.sqrt(b**2 - 4*a*c)
    # Choose sign to AVOID cancellation
    if b >= 0:
        x1 = (-b - disc) / (2*a)
    else:
        x1 = (-b + disc) / (2*a)
    # Use Vieta:  $x_1 x_2 = c/a$ 
    x2 = c / (a * x1)
    return x1, x2 # both accurate
```

✓ Vieta's formula ($x_1 x_2 = c/a$) avoids cancellation for the small root.



SECTION 3

Conditioning vs. Stability

The distinction students most consistently miss — and most frequently need.

Conditioning is a property of the PROBLEM. | Stability is a property of the ALGORITHM.

Conditioning of a Problem — Problem Sensitivity to Input Perturbations

A problem is well-conditioned if small changes in the input produce small changes in the output.

Condition Number $\kappa(f, x)$

$\kappa = (\text{relative change in output}) / (\text{relative change in input})$

Well-conditioned (κ small)

$f(x) = \sqrt{x}$ at $x=4$

A 1% change in x causes $\sim 0.5\%$ change in $\sqrt{x}$

$\kappa \approx 0.5 \leftarrow$ safe

Ill-conditioned (κ large)

$f(x) = x^{10}$ at $x=1.001$

A 0.01% change in x causes $\sim 0.1\%$ change

$\kappa = 10 \leftarrow$ amplifies errors by $10\times$

Catastrophically ill-conditioned



If a problem is ill-conditioned, NO algorithm can give an accurate answer — this is a fundamental mathematical limitation, not a coding issue.

Stability of an Algorithm — Resistance to Amplification of Round-off Errors

An algorithm is stable if the round-off errors introduced during computation do not grow catastrophically.

Forward Stable

The computed result $\tilde{f}(x)$ satisfies $|\tilde{f}(x) - f(x)| / |f(x)|$ is small.

The answer is close to the true answer. Straightforward but harder to guarantee.



Backward Stable

The computed result $\tilde{f}(x)$ is the exact answer for a slightly perturbed input: $\tilde{f}(x) = f(x + \Delta x)$ where $|\Delta x|/|x|$ is small ($\approx \epsilon_{\text{mach}}$).

The algorithm solved a nearby problem exactly. The gold standard in linear algebra.

Case Study: Polynomial Evaluation

```
• • • Python
# Naive:  $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ 
p_naive = sum(a[k] * x**k for k in range(n+1))
# Problems: many multiplications, accumulates errors

# Horner:  $p(x) = a_0 + x(a_1 + x(a_2 + \dots + x \cdot a_n))$ 
p_horner = a[-1]
for k in range(len(a)-2, -1, -1):
    p_horner = p_horner * x + a[k]
# Fewer ops AND more backward stable
```

Conditioning vs. Stability – The Critical Distinction

	STABLE Algorithm	UNSTABLE Algorithm
WELL-CONDITIONED Problem	Best case. Accurate result guaranteed. ✓ Accurate	Stable algo, hopeless problem. Result is best possible — but still inaccurate. △ Limited
ILL-CONDITIONED Problem	Good problem, bad algorithm. Switch to a stable algorithm! △ Fixable	Worst case. Bad problem AND bad algorithm. Errors compound. ✗ Garbage

Conditioning = the problem's fault. Stability = the algorithm's fault. You can fix the algorithm. You cannot always fix the problem.



L A B 1

Floating-Point Arithmetic in Practice

Precision · Machine Epsilon · Cancellation · Kahan Summation

Python 3.10+ · NumPy · Jupyter Notebook (.ipynb) | Submit: .ipynb + PDF export by Friday 11:59 PM

1

float32 vs float64 Summation Error

20 pts

~25 min

Sum $\sum(1/k)$ for $k=1..10^6$ in both precisions. Compare relative errors against a high-precision reference. Plot on a log-log scale.

2

Machine Epsilon — Manual & numpy

20 pts

~20 min

Implement a while-loop to find ϵ_{mach} by successive halving. Compare with `np.finfo()`. Explain the 2^{-52} discrepancy.

3

Catastrophic Cancellation Demo

20 pts

~25 min

Two examples: (A) quadratic formula, (B) $\log(1+x)$. Implement naive and stable versions. Quantify relative error before and after.

4

Kahan Compensated Summation

15 pts

~20 min

Implement `kahan_sum()` from scratch. Compare accuracy and timing with naive sum and `np.sum` for $N=10^6$ in float32.

+ Code Quality (10 pts) + Written Discussion (15 pts) = 100 pts total

Task 1 — Summation Error

• • • Python

```
from decimal import Decimal, getcontext
import numpy as np, matplotlib.pyplot as plt

getcontext().prec = 50 # high-prec reference
N = 1_000_000
exact = float(sum(Decimal(1)/Decimal(k) for k in
range(1,N+1)))

s32 = np.float32(0); s64 = np.float64(0)
for k in range(1, N+1):
    s32 += np.float32(1.0/k)
    s64 += np.float64(1.0/k)

err32 = abs(s32 - exact) / exact
err64 = abs(s64 - exact) / exact
print(f'float32 rel error: {err32:.3e}')
print(f'float64 rel error: {err64:.3e}')
```

Task 2 — Machine Epsilon

• • • Python

```
import numpy as np

def manual_eps(dtype):
    eps = dtype(1.0)
    while dtype(1.0) + eps != dtype(1.0):
        eps = eps / dtype(2.0)
    return eps * dtype(2.0) # restore last diff

for dtype in [np.float32, np.float64]:
    manual = manual_eps(dtype)
    numpy_eps = np.finfo(dtype).eps
    theory = 2.0 ** (-np.finfo(dtype).nmant)
    print(f'{dtype.__name__}:')
    print(f'  manual : {manual:.6e}')
    print(f'  numpy   : {numpy_eps:.6e}')
    print(f'  theory  : {theory:.6e}')
```

Expected output (Task 2):

```
float32:  manual=1.192093e-07  numpy=1.192093e-07  theory=1.192093e-07
float64:  manual=2.220446e-16  numpy=2.220446e-16  theory=2.220446e-16
```

Task 3 — Cancellation: log1p demo

```
• • • Python
import numpy as np
from mpmath import mp, log as mplog
mp.dps = 50 # 50 decimal places

for x in [1e-5, 1e-10, 1e-15]:
    exact = float(mplog(1 + mp.mpf(x)))
    naive = np.log(1.0 + x) # cancellation risk
    stable = np.log1p(x)    # numerically stable

    rel_naive = abs(naive - exact) / abs(exact)
    rel_stable = abs(stable - exact) / abs(exact)
    print(f'x={x:.0e}: naive={rel_naive:.2e} '
          f'stable={rel_stable:.2e}')
# x=1e-15: naive ~0.11 (11% error!) stable ~0
```

Task 4 — Kahan Compensated Summation

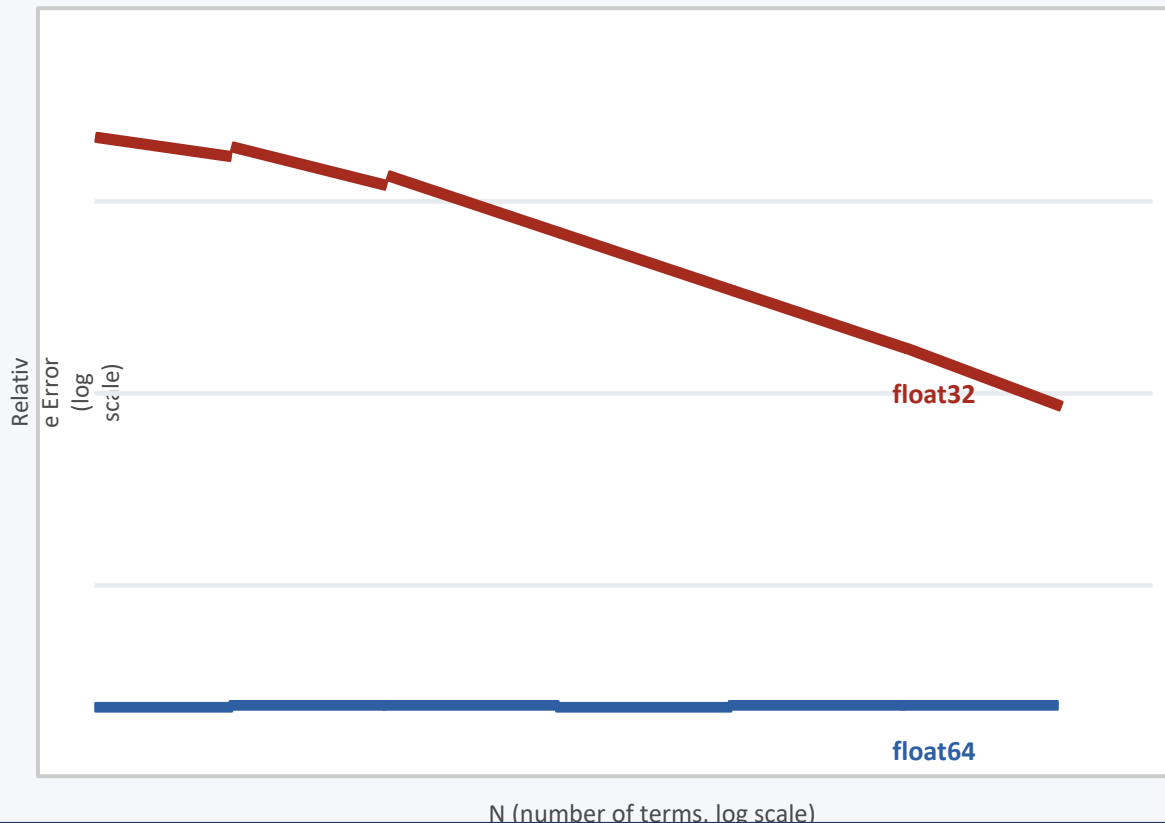
```
• • • Python
def kahan_sum(arr):
    """Kahan compensated summation.
    c tracks the running compensation for
    low-order bits lost each addition.
    """
    total = type(arr[0])(0.0)
    c = type(arr[0])(0.0) # compensation
    for x in arr:
        y = x - c          # corrected addend
        t = total + y       # new total
        c = (t - total) - y # bits lost
        total = t
    return total

# Compare accuracy: naive, Kahan, float64 ref
```

Key insight — Kahan summation:

- # The variable 'c' captures the rounding error from each step
- # and feeds it forward. Error grows as $O(\epsilon_{\text{mach}})$ instead of $O(n \cdot \epsilon_{\text{mach}})$

What your plot should look like:



float32 error

Grows with N. At $N=10^6$ expect $\sim 10^{-4}$ to 10^{-3} relative error. Dominated by accumulated rounding in low-order bits.

float64 error

Nearly flat. At $N=10^6$ expect $\sim 10^{-15}$ relative error. Extremely accurate despite 10^6 additions.

Gap between them

~ 9 orders of magnitude in accuracy. This is the cost of float32 for large summations. This is why float64 is the default in scientific computing.

```
Use plt.yscale('log') and  
plt.xscale('log')
```

Week 1 — Key Takeaways

1 Floating-point numbers are NOT real numbers — they are a finite approximation. $\sim 2^{64}$ double values exist in a space of uncountably many reals.

2 ϵ_{mach} is your precision floor. float32 gives ~ 7 decimal digits; float64 gives ~ 15 . Know which you are using at all times.

3 Distinguish four error types: absolute, relative, forward, backward. Backward-stable algorithms are the gold standard.

4 Catastrophic cancellation silently destroys accuracy when nearly equal values are subtracted. Always check, always reformulate.

5 Conditioning = problem property. Stability = algorithm property. Stable algorithms cannot rescue ill-conditioned problems.